

개인 프로젝트 시작 후 9.16~10.1 동안 저사양 온프레미스 환경(저사양 RAM, CPU 전용)에 최적화된 텍스트 전용 웹 애플리케이션으로 피벗 완료 후 리팩토링, 문서화, 형상 관리를 순차적으로 진행함. 우선 백엔드 서버(FastAPIUvicorn) 구동 환경 구축 및 패키지 구조 최적화를 진행.

주요 기술 Stack : Python 3.13, FastAPI, Uvicorn, SQLAlchemy, Pydantic

개인프로젝트 폴더 및 파일 구축 내용

src/service_ad_system/main.py

-역할: FastAPI 서버의 진입점(Entry Point). FastAPI()인스턴스를 생성하고, CORS 미들웨어 설정 및 각 API 라우터를 등록(app.include_router)하여 백엔드 서버를 구동합니다.

src/service_ad_system/api/ad.py

-역할: 클라이언트(프론트엔드)로부터 요청을 받아 처리하는 라우터 파일. @router.post("/generate")엔드포인트를 통해 사용자의 업종, 상품명, 타겟 정보를 받아 처리합니다.

src/service_ad_system/services/model_service.py

-역할: 4GB RAM / CPU 환경에 최적화된 Hugging Face Qwen2.5-0.5B-Instruct 모델을 직접 구동하는 핵심 AI 서비스 파일. 비동기 이벤트 루프(asyncio)와 파이프라인 설정을 통해 안정적인 텍스트 카피 생성 로직을 수행합니다.

src/service_ad_system/schemas/ad.py

-역할: Pydantic을 활용하여 API로 들어오는 입력 값과 나가는 출력 값의 데이터 구조와 타입을 엄격하게 검증(AdRequest, AdResponse)하는 스키마 파일입니다.

src/service_ad_system/core/database.py

-역할: SQLAlchemy를 이용해 로컬 SQLite 데이터베이스 연결 세션(get_db)을 생성하고 관리하는 설정 파일입니다.

src/service_ad_system/app.py

-역할: Streamlit을 활용하여 소상공인 사용자가 브라우저를 통해 직접 업종과 상품을 입력하고 실시간으로 AI 광고 카피를 확인할 수 있도록 만든 프론트엔드 대시보드 화면 UI 코드입니다.

(작업 내용)

시스템 아키텍처 및 디렉토리 구조 리팩토링 (src-layout)

작업 내용:

-유지보수성과 패키지 관리의 표준화를 위해 표준 src-layout 아키텍처로 전체 프로젝트 구조 재편하였습니다. 레거시 백엔드 디렉토리(backend/) 및 불필요한 이미지 생성/SNS 자동화 등 비핵심 모듈을 과감히 제거하여 경량화를 달성하였습니다.

명확한 관심사 분리(API 라우터, AI 모델 서비스, 프레임워크 코어, Streamlit UI)를 통해 코드 가독성 및 확장성 극대화하고자 하였습니다.

-저사양 하드웨어(RAM 4GB, CPU) 환경 맞춤형 AI 모델 파이프라인 최적화

초기 고성능/대형 모델(1.5B 이상) 구동 시 로컬 컴퓨터 RAM의 한계를 초과하여 Uvicorn 백엔드 프로세스가 에러 로그 없이 강제 종료(OOM Kill)되는 현상이 발생하였습니다.

해결 및 구현 (model_service.py):

허깅페이스 초경량 인스트럭트 모델인 Qwen/Qwen2.5-0.5B-Instruct로 모델 체급으로 전면 교체하였습니다 (메모리 점유율 1GB 대폭 축소).

파이프라인 호출 시 CPU 부하 완화를 위해 max_new_tokens=60 및 do_sample=False 설정 적용으로 추론 속도 및 안정성을 확보하였습니다.

비동기 이벤트 루프(asyncio, run_in_executor)를 도입하여 백엔드 API 블로킹 현상을 방지하였습니다.

-폴스택 웹 인터페이스 및 API 연동 구축

백엔드 (FastAPI):

비동기 처리 기반의 고성능 RESTful API 구현 (/api/v1/generate).

Pydantic을 활용한 엄격한 요청/응답 스키마(AdRequest, AdResponse) 검증 적용하였습니다.

프론트엔드 (Streamlit):

소상공인이 직관적으로 활용할 수 있는 반응형 웹 대시보드 UI를 구현하였습니다 (app.py).

업종, 상품명, 타겟층 입력에 따른 실시간 AI 광고 카피 및 해시태그 생성 기능을 연동하였습니다.

-문서화 및 형상 관리(Git/GitHub) 최적화

문서화:

상세한 기술 명세, 시스템 세팅 가이드, 그리고 트러블슈팅 내역을 담은 README.md를 작성하였습니다.

프로젝트 최종 보고서 PDF를 docs/ 디렉토리에 마운트하고 README.md 내에 직관적인 다운로드 및 조회 링크 연결하였습니다.

형상 관리 (Git):

.gitignore 설정을 통해 불필요한 빌드 캐시(__pycache__, *.egg-info, .vscode/)를 배제하고, 파이썬 패키지 구동에 필수적인 __init__.py 구조를 엄격히 유지하여 타 사용자 클론 시 구동 안정성을 확보하였습니다.

(트러블 슈팅 내역)

① 패키지 구조 및 절대 경로 전환 작업

문제 상황:

- 팀 프로젝트 구조에서 개인 프로젝트로 환경이 변경되면서 모듈 간 임포트 경로 불일치 발생 (ModuleNotFoundError).
- 특히 라우터 파일 내부에서 상대 경로(from ../database import get_db)를 사용할 때 윈도우 환경에서 패키지 인식 실패 문제 발생.

해결 내용:

- 프로젝트 내 모든 라우터 및 핵심 모듈의 임포트 방식을 상대 경로에서 명확한 **절대 경로 방식**(from service_ad_system.database import get_db)으로 전면 수정 및 코드 표준화 완료.

② 윈도우 환경 Uvicorn 리로드(--reload) 프로세스 경로 유실 문제 분석 및 해결

문제 상황:

- Git Bash 및 Windows 환경에서 PYTHONPATH=service_ad_system/src환경 변수를 지정하여 Uvicorn을 실행했으나, --reload옵션 사용 시 생성되는 자식

프로세스(SpawnProcess)가 부모의 환경 변수를 상속받지 못하고 유실됨.
-이로 인해 ModuleNotFoundError: No module named 'service_ad_system.database'에러가 반복적으로 발생.

원인 분석:

-윈도우 운영체제의 multiprocessing자식 프로세스 생성 특성상 외부 명령어 형태의 환경 변수 주입이 서브 프로세스에 온전히 전달되지 않는 한계 확인.

③ 실행 환경 최적화 및 원천 해결

해결 내용:

-외부 명령어 및 --reload옵션 의존성을 제거하고, 파이썬 코드 내부에서 sys.path.insert()를 활용해 src경로를 동적으로 강제 주입하는 방식을 도입.
-터미널 환경 변수 종속성을 없애고 윈도우/리눅스 어디서나 안정적으로 구동 되도록 실행 루틴 정립.

윈도우 환경에서의 경로 유실 및 ModuleNotFoundError원천 차단 완료

최상단 루트 폴더 기준 단일 명령어를 통한 안정적인 FastAPI 서버 구동 환경 구축 완료

로컬 개발 환경 동기화 및 형상 관리 체계 점검

작업 내용:

-VS코드 내 최신 수정 사항과 로컬 폴더 환경 간의 정합성을 맞추기 위해 Git 기반 커밋 및 원격 저장소 동기화 파이프라인 점검 (git add, git commit, git push).

-프로젝트 전반의 버전 관리 상태 확인 및 최신 상태 반영 완료.

레거시 코드 정리 및 중복 백엔드 서버 제거

진단 내용:

-백엔드 코드(backend/폴더 및 라우터 설정)를 진단하였고 해당 코드가 실제 AI 모델(ModelService)과 연동되지 않고 단순 더미 데이터를 반환하며, 패키지 구조상 충돌 위험이 있음을 확인.

조치 사항:

-중복 서버 구동 및 포트 충돌을 방지하기 위해 구버전 레거시 백엔드 폴더 및 파일 전면 삭제.

-신규 구현된 FastAPI 비동기 API 라우터와 model_service.py단일 체제로 백엔드 아키텍처 완전 일원화.

최종 보고서 PDF 연동 및 문서화 강화 (README.md)

작업 내용:

-프로젝트 루트 경로에 docs/디렉토리를 생성하고, 상세 기획 및 트러블슈팅 이 담긴 최종 보고서 PDF 파일(소상공인 맞춤형 AI 광고 서비스 (Service_AD_System) 프로젝트 보고서 이태훈.pdf) 배치.

-마크다운 링크 문법 및 웹 표준 URL 인코딩 규칙(%20공백 처리)을 적용하여 README.md내에 직관적인 다운로드 및 조회 링크 구현.

Git 형상 관리 최적화 및 .gitignore정밀 설정

문제 상황:

-로컬 개발 환경의 임시 파일, IDE 설정, 빌드 메타데이터가 깃허브 원격 저장소에 불필요하게 노출되는 문제 발생.

-파이썬 패키지 필수 구조인 __init__.py파일과 불필요한 캐시 파일 간의 관리 기준 정립 필요성 대두.

조치 및 구현:

-프로젝트 필수 패키지 구조 유지를 위해 __init__.py파일(내용은 비어있으나 패키지 인식을 위해 필수)은 깃허브 추적 대상에 정상 포함시킴.

- .gitignore 파일을 작성하여 불필요한 빌드 및 캐시 파일들이 원격 저장소에 올라가지 않도록 예외 처리 적용:

- IDE 설정: .vscode/
- 파이썬 빌드 메타데이터: *.egg-info/, 빌드 폴더
- 런타임 캐시: __pycache__/, 컴파일 파일
- 내부 학습/임시 리소스 폴더: src/service_ad_system/training/

- 레거시 제거 및 코드 경량화: 불필요한 구버전 백엔드 코드를 정리하고 단일화된 API 구조를 확립하여 시스템 안정성 확보.

- 표준 형상 관리 정립: 개발 환경 설정 파일과 필수 패키지 구조(__init__.py)를 명확히 구분하여, 타 사용자 클론 시에도 즉시 구동 가능한 재현성 높은 저장소 환경 구축.

- 프로젝트 문서화 완성: 기술 명세와 최종 보고서 PDF를 리드미에 유기적으로 연결하여 프로젝트의 완성도 및 가독성 극대화.